

ASP.NET Core CON C#

9. Helper HTML

La clase `HtmlHelper` genera elementos html de forma simple y puede enlazar un objeto del modelo a los elementos html(**binding**) para mostrar el valor de las propiedades del modelo en los elementos html y también asignar el valor de los elementos html a las propiedades del modelo al enviar el formulario web.

`Html.BeginForm` , `Html.EndForm`, `Html.ActionLink`
`Html.Hidden`, `Html.Label`, `Html.Password`
`Html.CheckBox`, `Html.RadioButton`, `Html.ListBox`
`Html.DropDownList`, `Html.TextArea`, `Html.TextBox`

@ **Html** es un objeto de la clase `HtmlHelper`. (El símbolo @ se usa para acceder al objeto del lado del servidor). `Html` es una propiedad de tipo `HtmlHelper` incluida en la clase base `WebViewPage`. Dicha clase tiene métodos `BeginForm`, `ActionLink`....

Por ejemplo:

- @using (`Html.BeginForm("acción", "controlador", FormMethod.Post)`)

genera la cadena `<form action="controlador/acción" method="post">`

Una de las ventajas de utilizar el helper es que no tengo que mapear la ruta

Hay muchos métodos de extensión para la clase `HtmlHelper` , que crea diferentes controles html.

La siguiente tabla enumera los métodos `HtmlHelper` y el control html que genera cada método.

HtmlHelper	Strogly Typed HtmlHelpers	Control de html
<code>Html.ActionLink</code>		Enlace de anclaje
<code>Html.TextBox</code>	<code>Html.TextBoxFor</code>	Caja de texto

HtmlHelper	Strogly Typed HtmlHelpers	Control de html
Html.TextArea	Html.TextAreaFor	TextArea
Html.CheckBox	Html.CheckBoxFor	Caja
Html.RadioButton	Html.RadioButtonFor	Boton de radio
Html.DropDownList	Html.DropDownListFor	Desplegable, combobox
Html.ListBox	Html.ListBoxFor	cuadro de lista de selección múltiple
Html.hidden	Html.hidden para	Campo escondido
Contraseña	Html.PasswordFor	Cuadro de texto de contraseña
Html.Display	Html.DisplayFor	Texto html
Html.Label	Html.LabelFor	Etiqueta
Editor html	Html.EditorFor	Genera controles HTML basados en el tipo de datos de la propiedad de la modelo especificada, por ejemplo, cuadro de texto para propiedad de cadena, campo numérico para int, doble u otro tipo numérico.

La diferencia entre llamar a los métodos HtmlHelper y usar etiquetas html es que el método HtmlHelper está diseñado para facilitar el enlace para ver datos o almacenar datos en los modelos.

HtmlHelper - TextBox

La clase HtmlHelper incluye dos métodos que crean una caja de texto `<input type="text">` en la vista: `TextBox ()` y `TextBoxFor ()`. El método `TextBox ()` es un método de escritura flexible, mientras que `TextBoxFor ()` es un método fuertemente tipado.

Html.TextBox(string name, string value, object htmlAttributes)

genera una caja de texto que puede almacenar en la misma el valor de una propiedad de un modelo que haya sido pasado a la vista desde el controlador. Si el primer parámetro no coincide con ninguna propiedad, se asume que ese el simplemente el id, name de la caja y no se muestra como valor de la misma.

Ejemplo:

```
@Html.TextBox("myTextBox", "This is value", new { @class = "control" })

<input class="control" id="myTextBox" name="myTextBox" type="text"
      value="This is value" />

@Html.TextBox("Nombre", null, new { @class = "control" })

<input class="control" id="Nombre" name="Nombre" type="text"
      value="" />
```

El valor que se muestra en la caja es el de la propiedad Nombre del objeto, por eso el segundo parámetro es null.

Por el contrario, si el método es usado en un envío al controlador el valor de la caja se asigna a la propiedad del modelo cargado en la vista cuyo nombre es ese.

Html.TextBoxFor es una caja tipada, sirve enlazar a un modelo tanto en operación de entrada como de salida de datos, es decir nos sirve tanto para almacenar el valor de la caja en la propiedad del modelo como para visualizar el valor de la propiedad del modelo en la caja

```
@Html.TextBoxFor(s=>s.Mensaje)
```

Ese método en una petición(Request) post al servidor actualiza el valor de la propiedad mensaje con lo que hayamos escrito en la caja. Por el contrario, si estamos retornando una vista a la que le pasamos un modelo, esa caja visualiza (propiedad value) el valor de la propiedad Mensaje del objeto que pasamos a la vista

Personalmente utilizo TextBoxFor para controlar que no me equivoco en el nombre de la propiedad

```
<div class="contacto">
    @using (Html.BeginForm()) //carga la misma url, es como refrescar la pág
    {
        @Html.Label("nombre", "Tu nombre:") @Html.TextBoxFor(n=>n.Nombre)
        @Html.Label("mensaje", "Mensaje:") @Html.TextBoxFor(n=>n.Mensaje) <br>
        <input type="submit" value="Aceptar" />
    }
</div>
```

HtmlHelper - CheckBox

La clase HtmlHelper incluye dos métodos de extensión para generar un `<input type="checkbox">`: `CheckBox()` y `CheckBoxFor()`.

`CheckBox(string name, bool isChecked, object htmlAttributes)`
`CheckBoxFor(expresión)`

Donde name puede ser el nombre de una propiedad booleana de un modelo. Al igual que con las cajas el parámetro puede funcionar como lectura o escritura de dicha propiedad.

```
@Html.CheckBox("Asistencia", true)
@Html.CheckBoxFor(obj => obj.Asistencia)
```

HtmlHelper - RadioButton

Ejemplo: Html.RadioButton () en Razor View

```
Male: @Html.RadioButton("Gender", "Male")  
Female: @Html.RadioButton("Gender", "Female")
```

Enviará al servidor la propiedad Gender de un modelo con el valor del radio que haya sido seleccionado.

Ejemplo: Html.RadioButtonFor () en Razor View

```
@model Student  
  
@Html.RadioButtonFor(m => m.Gender, "Male")  
@Html.RadioButtonFor(m => m.Gender, "Female")
```

Validación

En La vista utilizaremos otro helper **Html.ValidationMessageFor**, con una expresión lambda le indicamos la propiedad para la cual queremos mostrar sus mensajes de error (si los hubiese). Este helper nos proporciona algunos métodos:

- **ValidationMessageFor** para cada campo indicamos que el mensaje de error irá asociado a la validación de dicha propiedad del modelo

```
@using (Html.BeginForm()) {  
    @Html.ValidationSummary(true)  
    <fieldset>  
        <legend>DemoModel</legend>  
  
        <div class="editor-label">  
            @Html.LabelFor(model => model.Nombre)  
        </div>  
        <div class="editor-field">  
            @Html.EditorFor(model => model.Nombre)  
            @Html.ValidationMessageFor(model => model.Nombre)  
        </div>  
  
        <div class="editor-label">  
            @Html.LabelFor(model => model.Edad)  
        </div>  
        <div class="editor-field">  
            @Html.EditorFor(model => model.Edad)  
            @Html.ValidationMessageFor(model => model.Edad)  
        </div>  
  
        <p>  
            <input type="submit" value="Create" />  
        </p>  
    </fieldset>  
}
```

Los errores que se muestran son los que vienen predefinidos por la colección de errores y asociados a cada regla de validación que escribimos para las propiedades del modelo. Podemos personalizar dichos mensajes de dos formas:

1. A través del método del helper ValidationMessageFor

@Html.ValidationMessageFor(expresión lambda, "mensaje de error", estilos)

Existe el método análogo ValidationMessage que no utiliza expresión lambda y asigna la propiedad del modelo a la validación por coincidencia de nombre (primer parámetro que le pasamos)

2. Añadiendo dicho mensaje en la regla de validación de la propiedad del modelo

[Required(ErrorMessage="Please enter student name.")]

- **ValidationSummary:** método que genera una lista () con todos los errores de validación que se producen y se almacenan en ModelState. Podemos decidir mostrarlos todos juntos en formato de lista

• Nada de anónimos. ¡Aquí todo el mundo tiene un nombre!
• The Edad field is required.

DemoModel

Nombre

Nada de anónimos. ¡Aquí todo el mundo tiene un nombre!

Edad

The Edad field is required.

Create

Esa lista se mostrará en la posición en la que incluyamos en nuestra vista dicho método

@Html.ValidationSummary(true, "mensaje", estilos)

La cadena "mensaje" se mostrará antes de mostrar la lista. El primer parámetro por su parte indica si la lista debe mostrar todos los errores (true) o solo aquellos que no estén vinculados a ninguna propiedad (false),

Edit

Student

• The Name field is required.
• The Age field is required.

Name

Age

Save

[Back to List](#)

No debo tener asociado el método anterior `ValidationMessageFor` a ninguna propiedad

NOTA: Validación en cliente

La validación en cliente no tiene nada que ver con la seguridad, es un tema de usabilidad (darle feedback al usuario de forma más rápida) así que **el uso de validación en cliente no inhibe de realizarla en servidor**. ASP.NET MVC la realiza siempre en servidor (el desarrollador debe comprobar siempre el valor de la propiedad `IsValid` del `ModelState`).